Use Cases & End-to-End Flows

Five end-to-end scenarios — how one request becomes a reviewed PR, and how every run makes the system measurably better.

13 phases complete

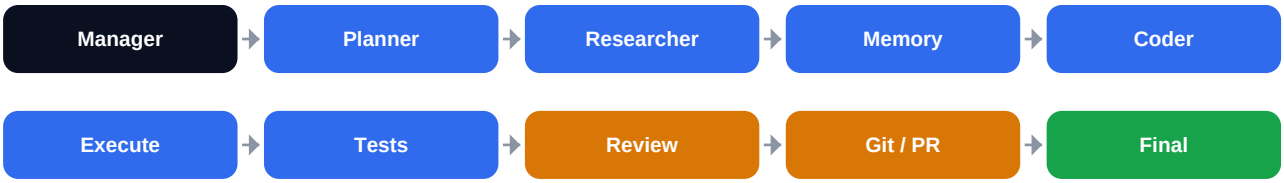
334 tests passing

runs fully offline

local models

The model behind every example

You submit one natural-language request. A **Manager** classifies and delegates it; an explicit workflow graph then sequences the specialists. Every agent reads from and writes to one shared *ProjectState* — they never call each other directly.



The part most tools skip

When the run finishes, the **Evaluation Engine** scores it (outcome + cost + which prompt versions ran), stores it in the **Performance Database**, and updates the **Analytics** dashboard. Failures are remembered; benchmarks track each release. **That** is what lets ForgeAI improve across runs without code changes.

Legend: ■ start ■ agent ■ gate / approval ■ end

1

BACKEND FEATURE

Add JWT authentication to an API

Who: a developer with a FastAPI app and no auth yet. **Goal:** protected routes with login/signup — reviewed, on a PR.

The request

```
POST /agents/run
{ "user_request": "Add JWT authentication", "project_id": "auth-1" }
```

Stage by stage

STAGE	WHAT HAPPENS	OUTPUT
Manager	Reads the request; **Dynamic Selection** classifies it as **backend** (signals: auth, jwt, api) and records the rationale.	*task_type = backend*
Planner	Breaks it into ordered tasks: add dep, hash passwords, issue/verify tokens, protect routes, register/login, tests.	*6 tasks*
Researcher	Pulls only the relevant context — routes, settings, the user model.	*scoped context*
Memory	Recalls past decisions ('we use argon2', 'tokens expire in 1440m').	*long-term recall*
Coder	Writes the auth module, token utils, and protected dependency from context.	*generated files*
Execution	Installs deps and builds inside an isolated Docker sandbox.	*build logs*
Testing	Runs the suite; reports pass/fail with evidence.	*tests passed*
Review	Checks security, naming, structure → APPROVED (else → Reflection).	*verdict: approved*
Git	Authors a commit on a local clone and **proposes** a PR — writes nothing yet.	*PR proposal (gated)*

You approve — then it's measured

■ The proposal lands in the **Approval Center**; you review the diff and click **Approve** → the PR is created. Nothing reached GitHub without your decision.

■ Recorded: an **Evaluation** (success=true, score=0.70, prompt_versions=v1) that now counts toward per-agent stats. A later merge becomes a positive *pr_accepted* signal.

2

SELF-CORRECTION

Fix a failing CI build — and never re-diagnose it

Who: a team whose pipeline broke. **Goal:** diagnose and fix — and never waste time on the same error twice.

First encounter — diagnose from scratch

STAGE	WHAT HAPPENS	OUTPUT
Coder → Execute	Code runs in the sandbox; build fails: ModuleNotFoundError: No module named 'jwt'.	*exit 1*
Testing	Tests fail — the import is missing.	*test_passed = false*
Review	Not approved → routes to Reflection (the retry loop).	*changes requested*
Reflection	Diagnoses the root cause and proposes a fix (install pyjwt); **stores it** in the Failure KB, keyed by a normalized signature.	*fix proposed + stored*
Coder (retry)	Applies the fix; Execute + Tests pass; Review approves.	*approved*

Next encounter — fixed instantly, no model call

Weeks later a different project throws the same error, buried in a long traceback. The signature normalizer collapses both to one key, so Reflection **recalls the known-good fix**:

```
error_signature('Traceback ...\nModuleNotFoundError: No module named \'jwt\'')
    ->  'modulenotfounderror:jwt'        # same key as the first time
recall('modulenotfounderror:jwt')
    ->  fix='pip install pyjwt'  outcome=RESOLVED    # reused, zero LLM cost
```

Why this matters

The system behaves like a **Stack Overflow for itself**. A fix that later fails is demoted, so the knowledge base self-corrects — no fix is trusted forever.

3

MULTI-AGENT DEBATE

Build a CRUD REST API — three plans, one winner

Who: a startup that needs an endpoint fast but done right. **Goal:** a well-scoped plan chosen from competing approaches.

```
POST /agents/run
{ "user_request": "Create a CRUD REST API for todo items",
  "project_id": "todos-1" }       # built with debate_planner = 3
```

The Planner debates instead of guessing once

Three **independent** attempts run from different angles — no attempt sees another's output:

STAGE	WHAT HAPPENS	OUTPUT
Attempt 0	*Simplest-increment* — ship a minimal create + list first.	*plan A*
Attempt 1	*Risk-first* — tackle validation, pagination, error paths early.	*plan B*
Attempt 2	*Thorough* — cover tests, docs, and error handling explicitly.	*plan C*
Judge (Review)	Scores candidates by a documented, deterministic rubric and records **which won and why**. The winner drives the pipeline.	*winner + rationale*

Recorded for the dashboard

The debate decision (winner + judge rationale) is on the run's audit trail; the Evaluation captures the score, so debated vs. non-debated runs can be compared over time. Debate is **off by default** and **deterministic** — same input, same winner — so it is safe and reproducible.

4

THE COMPOUNDING LOOP

A team that gets better over 100 runs

Who: any team running ForgeAI continuously. **Goal:** improvement that compounds — without anyone editing agent code.

STAGE	WHAT HAPPENS	OUTPUT
Run 1	Baseline. Scored and stored. Prompts at v1. Failures start filling the KB.	*score recorded*
Runs 2–50	Recurring errors fixed instantly from memory; scores + versions + outcomes accrue in the Performance DB.	*stats accumulate*
Prompt A/B	Someone registers Planner **v2**; runs split v1/v2; per-version stats are derived from the records.	*v1 vs v2 on dashboard*
Promotion gate	Once v2 has enough samples and clears the score margin, the system **recommends** promotion — requires approval, never automatic.	*gated recommendation*
Workflow-opt	If a task type succeeds uniformly without a step mattering, it **suggests** A/B-testing without it — advisory; the graph is never auto-edited.	*gated suggestion*
Benchmarks	Each release runs the versioned suite; the pass-rate trend shows real improvement, release over release.	*trend per version*

Why 100 > 1

Measurement (every run scored) + memory (failures reused) + comparison (versioned prompts & benchmarks) + gated learning means Run 100 is measurably stronger than Run 1 — and you can **prove** it on the Analytics dashboard, not just claim it.

Run the whole loop yourself

```
# offline, deterministic, no models needed — every Phase 12 component:
cd apps/api
PYTHONPATH="$PWD:$PWD/../../packages" \
  uv run python ../../scripts/demo_phase12.py
```

5

FROM NOTHING

First run: a working project in under a minute

Who: a brand-new user with an empty account. **Goal:** go from sign-in to a real, running project — no setup, no docs (Phase 13).

The onboarding flow

STAGE	WHAT HAPPENS	OUTPUT
Sign in	Register / log in; land on the **project chooser**, not a bare workspace.	*front door*
Choose a starter	Pick *Create New* and a starter — **empty**, or **fastapi-saas** (JWT auth, PostgreSQL, Docker, tests).	*starter selected*
Bootstrap	A real project directory is provisioned and the starter is scaffolded into it — deterministic, instant, offline (no model).	*files on disk*
Workspace opens	Bound to the new project, with suggested one-click first tasks so it's not a blank box.	*ready to build*
Run the team	Pick a suggestion (or type one); the agents extend the project and write files into it, live on the timeline.	*verdict + new files*

```
POST /projects/bootstrap   { workspace_id, name: 'My SaaS',
                             starter: 'fastapi-saas' }
   ->  scaffolds app/main.py, app/security.py, tests/, Dockerfile, ...
POST /agents/run           { user_request: 'Add a /status endpoint',
                             project_id: <new project> }
   ->  team runs, writes files into the project, run is scored
```

The first-minute wow

The entire path — register → bootstrap → run → scored in analytics — works **offline with MODEL_PROVIDER=echo**, so the first-minute experience is demoable on any hardware and is covered end-to-end by the test suite.

13 phases complete · 334 tests passing · 26 ADRs · fully documented · runs on local models.